DEEP PACKET ANALYZER

Project explanation + interview preparation

Architecture | Low-level design | System design | 80 answered questions

A practical guide based on your current repository.

Use this to understand the implementation, explain your decisions, and handle follow-up questions. Answers use simple language and distinguish current behavior from proposed improvements.

Prepared: 30 September 2026

Scope: C++17 engine, Npcap/PCAP input, Express API, local JSON telemetry, and the vanilla JavaScript dashboard. No guide can predict every interview question; this covers the main project-specific areas and common follow-ups.

How to use it

First practise the 30-second explanation. Then draw the architecture without looking. Study questions 1-30 for a first interview, 31-60 for implementation depth, and 61-80 for design and debugging. Finish by demonstrating one complete packet-to-dashboard path.

Honest answers are stronger

The current system does not decrypt HTTPS, implement TCP stream reassembly, provide a distributed service, or measure classification accuracy against independent labels. Saved rules, loaded rules, and verified operating-system enforcement are separate states.

Reading map

Section	What to prepare
Project explanation	30-second pitch, 2-minute pitch, example packet journey
Architecture	Processes, threads, data flow, configuration and telemetry
Low-level design	Classes, interfaces, ownership, data structures and invariants
System design	Current constraints and a proposed multi-sensor design
Questions 1-10	Project value and networking basics
Questions 11-20	Classification and high-level architecture
Questions 21-30	C++ design and concurrency
Questions 31-40	Data structures and rules
Questions 41-50	API, reliability and frontend
Questions 51-60	Testing, performance and security
Questions 61-70	Scaling and design trade-offs
Questions 71-80	Debugging, ownership and interview practice
Final pages	Demo script, revision sheet, evidence and source map

Answer pattern

Give a direct answer, connect it to a class or file, explain one trade-off, and state one limitation. For example: "I use a bounded queue between capture and analysis. PacketQueue protects it with a mutex and condition variable. It absorbs short bursts, but when it fills, new packets are dropped and counted."

Explain your project simply

One sentence

DeepPacket Analyzer is a local tool that reads network packets, extracts useful metadata, groups packets into flows, evaluates rules, and displays the results in an interactive dashboard.

30-second interview pitch

My project has a C++ packet-analysis engine and a web dashboard. It works with live network capture or saved PCAP files. The engine parses packet headers and uses visible DNS, HTTP Host, or TLS SNI information to identify traffic. It tracks flows, evaluates filtering rules, and exports bounded JSON telemetry. An Express server serves the dashboard, where users can filter packets, inspect headers, explore connections, and manage rules.

Two-minute explanation

The problem is that raw packets are difficult to understand. I split the solution into capture, analysis, and presentation. Live capture uses a producer thread and a bounded queue; a consumer parses and classifies packets. A connection tracker combines traffic in both directions and evaluates rules. A telemetry thread exports snapshots, which the Express API serves to a browser. The interface shows traffic timelines, protocols, top talkers, a connection graph, and packet details. I also support offline PCAP analysis for repeatable testing. Improvements include proper JSON configuration loading, reliable PCAP headers, bounded metadata retention, responsive tables, and automated browser checks. Current limitations include no TLS decryption or TCP reassembly, and Windows firewall enforcement still needs a controlled elevated verification.

A simple analogy

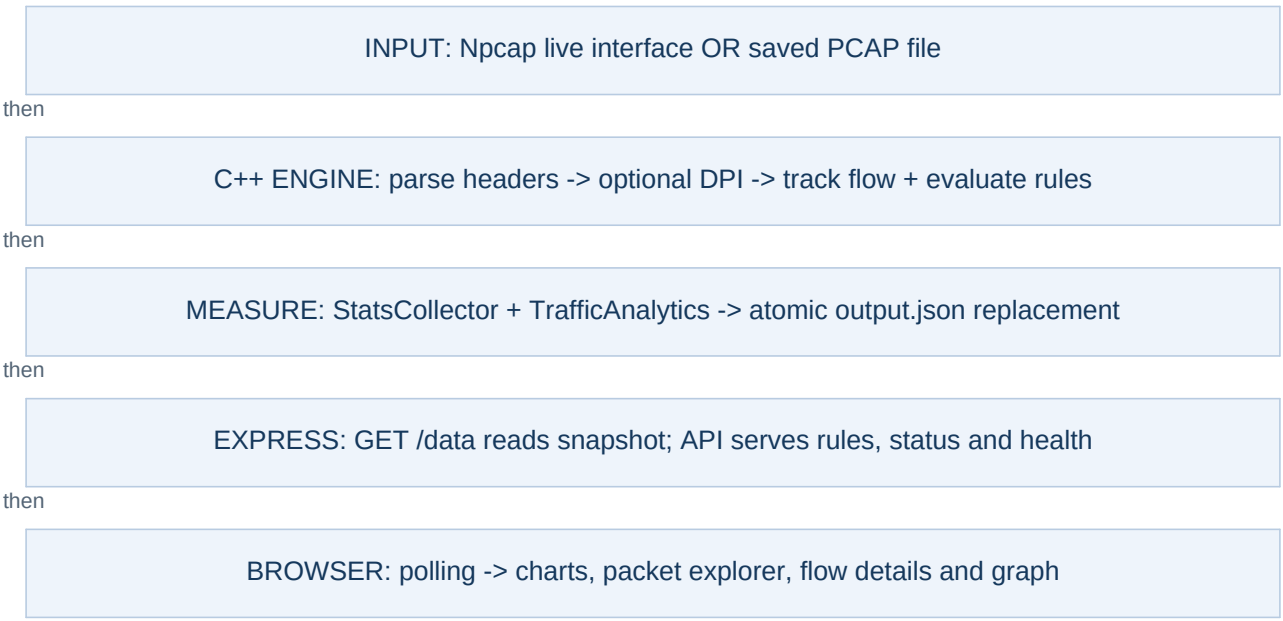
Think of packets as parcels. Header parsing reads the address label. A flow groups parcels from the same conversation. Classification uses visible clues about the service. Rules mark policy decisions. The dashboard is a summary desk; it does not retain every parcel or reveal encrypted contents.

What makes it a final-year project?

It combines systems programming, network protocols, concurrency, APIs, UI design, reliability, and testing. Its strongest interview value is the reasoning you can explain and the evidence you can reproduce. A polished screen alone is not proof of accuracy or production readiness.

Architecture: the current system

Draw this first on a whiteboard.



Two additional paths

Control path: browser rule write -> authenticated Express handler -> rules.json + revision notification -> C++ reload -> acknowledgment. Optional export path: packets with FORWARD decisions -> filtered PCAP file. Windows WFP is an optional OS filtering integration, separate from passive capture.

Process boundary

The C++ executable and Node server are separate processes. The browser communicates with Node over HTTP. Node and C++ exchange local files; there is no current message broker, database, or direct browser-to-engine RPC.

Runtime and packet sequence

Execution role	Current responsibility
Live producer	Read raw frames and move them into PacketQueue.
Live consumer	Parse, optionally inspect, track, evaluate, update statistics, optionally export.
Telemetry thread	Periodically produce snapshots, approximately every 500 ms.
Rule reload thread	Check notification/revision approximately every 500 ms; validate and acknowledge successful reload.
Offline mode	Sequential packet processing; rule reload thread still exists. Final telemetry is exported.

Example: a visible HTTP request

A captured Ethernet frame contains an IP packet, a TCP segment, and an HTTP request with a Host header. PacketParser extracts boundaries and headers. FastPath allows inspection for payload on port 80. DPIEngine extracts visible HTTP evidence. ConnectionTracker finds the forward or reverse five-tuple and evaluates rules. StatsCollector records counters and TrafficAnalytics retains metadata. Express returns a snapshot and the browser displays it.

Failure paths

A malformed frame is rejected and increments parse_errors. A full live queue increments processing_drops. Missing or invalid telemetry gives /data an HTTP 503 response. A malformed replacement rules file must not clear the last valid rules. A missing chart library must not disable the entire interface.

Time has three meanings

Capture timestamp describes when the frame was recorded. generated_at describes when the engine wrote telemetry. Processing elapsed time measures how quickly analysis ran. An offline file can be processed quickly even if its captured traffic covers several minutes.

Low-level design: class map

Class / module	Main responsibility and relationship
PacketSource	Abstract open(), close(), getNextPacket(). PcapFileSource and LiveCaptureSource provide input implementations.
PacketQueue	Owns queued RawPacket objects; bounded FIFO; mutex, condition variable and atomic counters.
PacketParser	Converts RawPacket into ParsedPacket with checked header and payload boundaries.
FastPath / DPIEngine	Eligibility check followed by stateless inspection. FastPath is not a classification cache.
ConnectionTracker	Owns flow state and RuleManager; correlates DNS; emits observations and recent flow records.
RuleManager	Owns rule sets/patterns; shared mutexes protect reads and mutations; validates configuration.
StatsCollector	Protects counters and owns TrafficAnalytics; exports snapshots.
TrafficAnalytics	Retains bounded packet metadata, buckets, histogram, endpoints and port aggregates.
WfpEnforcement	Optional Windows filtering integration; separate from a capture-policy decision.
Frontend modules	app.js orchestrates; charts, explorer, network, rules and DOM helpers separate UI concerns.

Ownership and lifetime

RawPacket owns the captured bytes. ParsedPacket may refer into that buffer during processing. Retained analytics must not keep a payload pointer after the buffer dies. PacketSource is selected through unique ownership. Threads must finish before the objects they access are destroyed.

Low-level design: contracts and limits

Contract / structure	Meaning
FiveTuple	Source IP, destination IP, source port, destination port, transport protocol. reverse() supports bidirectional lookup.
AppClassification	App category, evidence method, observed name, and optional HTTP metadata.
PacketAction	The pipeline mainly uses FORWARD or DROP. These are policy outcomes, not proof of network delivery or denial.
Rule status	Saved revision and applied revision distinguish persisted configuration from an engine acknowledgment.
Snapshot	schema_version 1 with additive fields, session_id, generated_at, counters, flows and analysis.
Bounded retention	Queue 10,000; tracked flows 10,000; recent flow records 100; packet metadata 300; timeline 300 seconds.
Aggregate limits	At most 4,096 endpoint keys and 4,096 port keys; top 20 exported. Limit flags mark incomplete inventories.

Useful invariants

Never read beyond captured bytes. Never keep a dangling payload pointer. Never replace valid rules with a partially parsed config. Never label missing history as zero traffic. Reset browser rate calculations after a session change. Do not count a policy match as an Npcap or queue drop.

An important consistency limitation

Atomic file replacement prevents a reader from seeing a half-written JSON file. It does not make all separately collected counters and tracker views one globally transactional measurement. A future unified snapshot API or versioned immutable snapshot would improve cross-field consistency.

System design: current versus proposed

Concern	Current design	Proposed extension
Deployment	Local engine + local Node API	Many authorized sensors + central ingestion
Transport	Atomic JSON file and HTTP polling	Batched telemetry with backpressure; SSE for browser updates
Persistence	Latest bounded snapshot	Historical database + controlled PCAP object storage
Identity	Localhost default; token for rule writes	Authenticated users, sensor identity and role-based access
Scaling	One consumer; bounded state	Flow-affine worker shards and sensor partitioning
Reliability	Counters, status, revisions	Sequence IDs, retry policy, deduplication and explicit loss accounting

Start by asking requirements

How many sensors? What traffic rate? Is payload storage allowed? How long must history remain? What latency is acceptable? Must rules enforce traffic or only mark observations? What are the storage and privacy constraints? Establish these before choosing infrastructure.

Capacity example: an assumption, not a benchmark

If 10 sensors each export 2,000 metadata records per second at 200 bytes per record, ingestion is 4 MB/s before protocol overhead. Over a day that is about 346 GB before indexing, replication, or compression. This motivates aggregation, retention, and optional sampling; it is not a measurement of this project.

Order of implementation

First add reliable local history and clear retention controls. Then improve snapshot consistency and performance profiling. Add multi-user identity and auditing before remote administration. Add distributed ingestion only when measured demand justifies the operational cost.

Project fundamentals

Interview questions 1-5

01. What problem does the project solve?

It turns raw packet traffic into understandable observations: who communicated, which protocols appeared, how much traffic was seen, and which configured rules matched. The dashboard reduces the need to read individual packets manually while retaining an inspector for details.

02. Who is the intended user?

A student, developer, or administrator investigating traffic on a machine or analyzing an authorized PCAP. The current deployment is local. It is not yet a centrally managed enterprise monitoring platform.

03. Why use C++ and Node together?

C++ handles packet buffers, parsing and native capture integration. Node provides a small HTTP API and static UI hosting. The boundary keeps browser concerns out of the packet loop, at the cost of coordinating two processes and a file-based contract.

04. How is this different from Wireshark?

This project focuses on a smaller custom workflow: application evidence, policy status, bounded analytics and a tailored dashboard. Wireshark is a much broader protocol analysis tool. I would describe this as a focused educational system, not a replacement with equivalent protocol coverage.

05. What is the strongest technical part?

Choose the part you can demonstrate: the capture-to-analysis pipeline, safe parsing, bidirectional flow tracking, or reliable rule reload. Explain a concrete failure and fix. For example, invalid JSON must be rejected before replacing working rules. Only claim personal ownership of work you understand and actually performed.

Networking foundations

Interview questions 6-10

06. What is a packet, and what is a flow?

A packet is one unit of network-layer traffic; capture commonly provides the enclosing link-layer frame. A flow groups related traffic using addresses, ports and protocol. This project tracks both directions together, so a request and response can update the same conversation.

07. What is the five-tuple?

Source IP, destination IP, source port, destination port and transport protocol. For example, 192.0.2.10:51000 to 198.51.100.20:443 over TCP. Reversing the addresses and ports describes the response direction. Ports alone do not identify the application.

08. How do TCP and UDP differ?

TCP provides an ordered byte stream with reliability and connection state. UDP carries independent datagrams without those guarantees. The analyzer reads their headers, but it does not implement TCP stream reassembly or guarantee that all bytes of an application message are visible.

09. What are DNS, HTTP Host and TLS SNI?

DNS can associate a name with an address. An HTTP Host header identifies the requested host in visible HTTP traffic. TLS SNI can expose a requested server name in a visible ClientHello. These are classification clues, not access to encrypted application content.

10. Why distinguish captured length from original length?

A capture can contain only a prefix of the original frame. Parsing must use available captured bytes to avoid an out-of-bounds read. Traffic volume can use original frame length when supplied. The inspector exposes both so a truncated frame is not mistaken for a complete one.

Classification and its limits

Interview questions 11-15

11. How does the engine identify applications?

FastPath selects eligible payloads on ports 80, 443 or 53. DPIEngine inspects visible protocol evidence, while ConnectionTracker can correlate DNS names and map domains to friendly application labels. Port-only classifications are labeled as such; domain mappings are heuristics.

12. Is FastPath a cache?

No. In this repository it checks whether payload exists and whether either port is 80, 443 or 53. It avoids unnecessary inspection, but does not look up a previously classified flow. A genuine classification cache would require keys, expiry and invalidation.

13. Can the project decrypt HTTPS?

No. It can use visible metadata such as a server name when available, or a port-based fallback. The encrypted HTTP body is not available from that metadata. Missing, fragmented or protected handshake information can reduce identification.

14. Why can DNS correlation be wrong?

Several domains may share an address, mappings may change, and a capture may miss the relevant DNS response. A TTL helps expire old mappings but does not prove which domain a later connection intended. Treat correlation as evidence with limitations.

15. What happens if a message spans TCP segments?

The current implementation inspects packet payloads without reconstructing the TCP byte stream. A split HTTP header or TLS ClientHello can be missed. A future reassembler needs sequence tracking, bounded buffering, timeouts, retransmission handling and careful testing before classification improves reliably.

Architecture decisions

Interview questions 16-20

16. Explain the end-to-end data flow.

A source yields a raw frame. PacketParser extracts headers; FastPath may invoke DPIEngine. ConnectionTracker updates flow state and policy. StatsCollector updates analytics and exports JSON. Express serves the snapshot and the browser presents it through charts and tables.

17. Why support offline and live modes?

Live mode shows current observations. Offline mode gives reproducible inputs for debugging, demonstrations and tests. Both reuse the same parsing and classification logic. Live mode adds capture concurrency and drop monitoring; offline processing speed is not the original network traffic rate.

18. Why separate telemetry from packet analysis?

Writing JSON and serving a UI should not be the responsibility of the capture callback. A separate exporter reduces coupling, although locks and serialization still have costs. Exporting bounded snapshots also prevents dashboard size from growing with every packet.

19. Why use JSON files instead of a database?

The current workload is a local latest-snapshot dashboard. JSON is easy to inspect and keeps installation small. It lacks historical queries, multi-writer transactions and strong distributed coordination. Those requirements would justify a database or another IPC design.

20. What is the data plane versus the control plane?

The data plane captures, parses and measures packets. The control plane changes rules and communicates revision status. Keeping their responsibilities distinct makes failures easier to explain: a saved configuration is not yet a confirmed engine load or a verified OS filtering result.

C++ and object design

Interview questions 21-25

21. Where is polymorphism used?

PacketSource is an abstract interface with a virtual destructor and virtual source operations. LiveCaptureSource and PcapFileSource supply different implementations. The processing pipeline works through the interface, making another source possible without rewriting the core packet logic.

22. Why use unique_ptr for a source?

It expresses that one owner is responsible for the source object and its lifetime. Moving the pointer transfers ownership. The virtual destructor supports correct derived destruction. Resource wrappers and explicit shutdown remain important for native handles and running threads.

23. Why use std::optional for classification?

Some packets do not undergo deep inspection. An optional value distinguishes no classification result from a populated classification object. That avoids inventing evidence for packets such as empty acknowledgments, although the flow may still have earlier classification information.

24. What does std::move do in the queue?

It allows the packet object and its owned buffer to be transferred into and out of the queue instead of unnecessarily copying the buffer. It does not itself make code thread-safe or guarantee zero copies throughout the capture pipeline.

25. What is a dangling pointer risk here?

ParsedPacket may hold a pointer into RawPacket payload storage. If analytics kept that pointer after RawPacket was destroyed, later access would be invalid. Retained telemetry keeps metadata and clears payload references; packet bytes are only inspected while their owner is alive.

Concurrency and synchronization

Interview questions 26-30

26. Why use a producer-consumer queue?

Capture and analysis have different timing. A queue absorbs short bursts so capture need not wait for every analysis operation. It cannot solve sustained overload: if incoming traffic consistently exceeds processing capacity, the queue fills and loss must be reported.

27. How is PacketQueue thread-safe?

A mutex protects the FIFO and related shared state. A condition variable wakes a waiting consumer when data arrives or shutdown occurs. Some counters and the stop flag are atomic. Atomic counters do not remove the need to protect the queue itself.

28. Why wait with a predicate?

A condition variable may wake without usable work. The predicate checks whether data exists or shutdown was requested. The consumer then verifies state before popping. This avoids busy waiting and prevents assuming that every wakeup means a packet arrived.

29. What happens when the queue is full?

push rejects the new packet and increments processing_drops. Existing queued work remains available. This is a deliberate bounded-memory trade-off, not guaranteed lossless capture. Capture-driver drops are counted separately, so two different loss sources remain visible.

30. How would you add more consumers safely?

First profile to identify the bottleneck. Then assign both directions of a flow to the same worker using a canonical flow key. Give workers separate state where possible and merge aggregates carefully. Adding threads without flow affinity can increase locking, reorder processing and corrupt assumptions.

Data structures and complexity

Interview questions 31-35

31. Why use an unordered_map for flows?

It supports average constant-time lookup by FiveTuple. The tracker looks for forward and reverse tuples so both directions can share state. Worst-case lookup is linear, and practical cost also includes hashing, allocations and synchronization. Do not claim the whole packet pipeline is constant-time.

32. Why use sets for exact rules and a vector for patterns?

An unordered_set supports average constant-time membership for exact IP, app, domain or port rules. Wildcard patterns need matching logic and are scanned. If there are R patterns, this stage can grow with R and domain length; it is not the same cost as exact lookup.

33. Why retain only 300 packet records?

A fixed recent window makes memory use and browser rendering predictable. The trade-off is that old packets disappear from the dashboard. An offline source PCAP may still contain them, but live dashboard metadata is not a complete historical archive.

34. How is a traffic timeline different from a histogram?

The timeline groups counts and bytes by captured second, preserving a time dimension within its retained window. The size histogram groups all parsed frames into six length ranges. A timeline gap means missing observations for that interval, not a fabricated measurement.

35. What are the major memory costs?

Queued raw frame buffers, retained flow state, strings, DNS correlations, packet metadata and aggregate maps. A 10,000-packet queue is an item limit, not a fixed byte limit. Long sessions also warrant checking every auxiliary cache and testing actual peak memory.

Rules and enforcement

Interview questions 36-40

36. What rule types are supported?

Source IPv4 address, destination port, application category and domain patterns. RuleManager holds exact sets and domain patterns and returns a block reason. Domain normalization and suffix boundaries matter so unrelated names are not matched accidentally.

37. How does hot reload work?

The API validates a request, writes rules and a revision, and creates a reload notification. The engine checks the flag and revision, validates the new JSON, and acknowledges a successful load. Revision polling helps recover when notification handling overlaps another save.

38. What does saved versus loaded mean?

Saved means the API persisted a rules revision. Loaded means the engine acknowledged the matching revision. Neither state alone proves that Windows blocked a specific connection. UI status should show each stage rather than treating an HTTP success as enforcement proof.

39. Why validate the whole configuration before replacement?

Partially parsed configuration could remove valid policy or leave mixed state. Build and validate replacement containers first, then install them under appropriate locking. If parsing fails, retain the previous working rules and report the failure.

40. Does a DROP decision stop live network traffic?

Not in passive monitor mode. It records a policy match; optional filtered PCAP export excludes matching packets. Actual Windows filtering uses the separate WFP integration when protection is requested and initialization succeeds. The recorded smoke test did not verify elevated WFP denial.

API and persistence

Interview questions 41-45

41. What are the main endpoints?

GET /data serves telemetry; GET /mode serves source and status information. /rules supports reads and authenticated POST/DELETE changes. /rules/status reports revision state. /critical-websites and /health support the website view. Static assets include a locally served Chart.js bundle.

42. Why use atomic file replacement?

Write a complete temporary file, then replace the destination so a reader sees an old or new file rather than a partially written one. This does not make several related files a single transaction, and it is not by itself a guarantee of crash durability.

43. How are unavailable and stale data different?

Unavailable means usable telemetry could not be read; /data returns 503. Stale means a snapshot exists but is too old or lacks a valid timestamp. Offline/stopped data is another state. The UI should display the distinction instead of presenting old values as live.

44. Why cache website health checks?

Many dashboard requests should not repeat identical outbound probes. The health module shares in-flight work, limits concurrent probes to four, uses a four-second deadline and caches results for ten seconds. Rule/configuration changes influence cache selection.

45. What is the weakness of synchronous file reads in Node?

They block the event loop while I/O completes. For small local snapshots this simplifies the implementation, but under slow disk or many clients it can delay other requests. A proposed improvement is an asynchronously refreshed validated snapshot cache, measured before and after the change.

Frontend and user experience

Interview questions 46-50

46. Why keep vanilla JavaScript?

The current UI is manageable with focused modules and does not require a framework to render bounded tables and charts. This limits build complexity. A framework could help if state and routing become much more complex, but adopting one alone would not improve correctness.

47. How is browser work kept bounded?

Requests do not overlap, hidden tabs poll less frequently, unchanged snapshots avoid unnecessary rendering, and tables use pagination. Graph size and metadata arrays have limits. These choices reduce DOM work, but large future history queries would need server-side filtering and pagination.

48. What does Pause updates do?

It freezes browser refreshes only. The capture engine can continue processing. A real start/stop capture feature would require a separate authorized process-control API with lifecycle management; the current UI does not claim to have that feature.

49. How are packet rate and traffic volume calculated?

Live packet rate comes from counter deltas over telemetry timestamp deltas, resetting for a new session or resume. Frame volume uses length measurements. The capture timeline uses packet timestamps. These represent different quantities and must not be mixed with offline benchmark speed.

50. What does the connection graph prove?

It shows communication between endpoints in recent flow records, limited to 18 hosts and 40 links. Node or edge weights reflect observed traffic. Node placement does not establish physical topology, device identity, or a router role. It is an exploration view of retained evidence.

Testing and measurement

Interview questions 51-55

51. How did you test the project?

The recorded verification includes 14 engine/API/analytics checks and 7 browser scenarios. Tests use synthetic PCAP fixtures and exercise rules, hostname retention, telemetry, parsing outcomes, export and UI interactions. A separate short Npcap smoke check observed real packets and a rule acknowledgment.

52. Why use synthetic PCAP fixtures?

They give known repeatable headers, traffic mixes and expected counts without putting personal captures in source control. The engine processes the frames to produce results. Synthetic fixtures verify selected behavior, but cannot establish general accuracy on diverse real-world traffic.

53. How do you test a PCAP writer?

Write a filtered PCAP, then reopen and process it. Check that its global header is valid and expected forwarded packets remain. This catches the earlier uninitialized-header defect more meaningfully than merely checking that an output file exists.

54. What performance can you honestly quote?

The recorded Release mixed-fixture run processed 100,000 packets in 2,150 ms, about 46,510 packets per second. It includes process startup, shutdown and telemetry but excludes fixture generation. It is one local offline benchmark, not a guaranteed live throughput or independent accuracy result.

55. How would you measure classification accuracy?

Build an independently labeled dataset covering known and unknown traffic, different applications and difficult captures. Report precision, recall, coverage and unknown rate by class. Separate direct name evidence from heuristic labels. Do not train or tune mappings and evaluate them on only the same examples.

Security and privacy

Interview questions 56-60

56. How are rule writes protected?

The server binds to localhost by default and requires an x-rule-token value for mutations. It validates rule values and limits JSON request size. This is a local control model; remote multi-user operation would require stronger identity, authorization, transport protection and auditing.

57. Why is metadata still sensitive?

Addresses, domains, timestamps and communication patterns can reveal user activity even without payload bytes. Keep captures authorized, restrict access and plan retention. The dashboard intentionally does not retain raw payload, but optional PCAP files can contain sensitive content.

58. How do you avoid unsafe UI rendering?

Treat packet-derived strings as untrusted data and escape them before insertion into HTML. Validate input server-side as well. CSV exports also neutralize formula prefixes because spreadsheet applications can interpret certain cell values as formulas.

59. Are alerts proof of an attack?

No. They are heuristic observations. The UI should preserve supporting evidence and avoid stating that a high count or pattern proves malicious behavior. Real detection claims require labeled evaluation, false-positive analysis and operational context.

60. What would you review before exposing the API remotely?

Read authorization, token handling, TLS, access boundaries, request limits, health-probe destinations, audit logs and data retention. Current telemetry reads are designed for a local deployment. Binding to another address alone does not create a safe multi-user service.

Scaling the system: proposed work

Interview questions 61-65

61. How would you support multiple machines?

Run an authorized capture agent on each machine, attach sensor and session IDs, and send bounded batches to an ingestion service. Store history centrally and serve aggregate queries. Keep capture working through short ingestion outages with an explicit bounded buffer and loss policy.

62. Would you choose WebSockets or SSE?

If the browser mainly receives telemetry, SSE is a simpler proposed push channel. WebSockets fit bidirectional streaming requirements. Current HTTP polling is sufficient for a local one-second dashboard. Choose based on latency, number of clients and recovery behavior, not fashion.

63. Which database would you add first?

For a single-machine history feature, SQLite is a reasonable design candidate because it is embedded and transactional. For shared multi-user queries, consider a server database after defining access patterns. Raw PCAP files should not automatically become large relational rows; use references and retention controls.

64. How would you handle duplicate telemetry after retries?

Use stable sensor ID, session ID and batch sequence numbers. Make ingestion idempotent or deduplicate by that identity. Track missing sequences separately. This is a proposed distributed design; the current local snapshot exchange does not implement a durable event-delivery protocol.

65. How would you control storage growth?

Separate recent detail from long-term aggregates. Apply time-based retention, quotas and optional sampling with visible loss indicators. Delete or expire related indexes and files consistently. Before implementing, estimate ingestion bytes per second and multiply by retention, replication and index overhead.

Design trade-offs and deeper follow-ups

Interview questions 66-70

66. Is the current snapshot fully consistent?

It is a complete JSON file, but tracker getters and statistics may be sampled at slightly different moments. File atomicity is different from application-wide snapshot consistency. A proposed coordinated snapshot or immutable versioned state would make cross-field relationships easier to guarantee.

67. How would you prevent deadlocks as the design grows?

Document lock ownership and acquire multiple locks in a consistent order. Keep critical sections small and avoid slow I/O while holding shared state locks where possible. Prefer immutable snapshots across component boundaries. Adding atomics to some fields does not prove deadlock freedom.

68. Why not just increase the queue size?

A larger queue absorbs longer bursts but consumes more memory and can increase analysis latency. It does not fix sustained overload. Measure capture rate, service rate, queue depth and loss, then improve the bottleneck or adopt an explicit sampling policy.

69. How would you improve domain classification?

Add representative labeled captures, stronger protocol parsing, bounded stream reassembly and evidence-aware mappings. Track unknowns and ambiguous shared addresses. Measure the improvement before claiming success. Adding more domain strings alone may increase apparent coverage while also increasing wrong labels.

70. What would you refactor first?

I would evaluate a shared processPacket operation to reduce duplication between live and offline loops, and a unified tracker snapshot API to improve consistency. Keep behavior covered by existing tests. These are proposed improvements, not claims that the current code already has those abstractions.

Debugging scenarios

Interview questions 71-75

71. The dashboard is stale. What do you check?

Check engine_state, generated_at and source mode, then whether the engine is running and writing to the expected output path. Confirm /data response and browser polling. Distinguish an intentionally completed offline capture from a stalled live engine before changing code.

72. Packets are missing. How do you locate the loss?

Compare capture_drops, processing_drops and parse_errors. Also distinguish dashboard retention of 300 records from capture loss. The UI may show only the latest metadata even when counters include many more frames. Validate with a known fixture before drawing conclusions.

73. A rule says saved but has no effect. Why?

Inspect the saved revision, applied revision and engine status. Confirm the engine reads the same rule location and accepts the JSON. Then determine whether the expected outcome is a policy label, a filtered export, or actual WFP enforcement; they require different evidence.

74. An application label looks wrong. What do you inspect?

Check the evidence method, observed domain, port and associated flow. Determine whether the label came from direct metadata, DNS correlation or a fallback mapping. Compare against known traffic and preserve ambiguous results as unknown rather than inventing certainty.

75. The UI is slow after adding history. What changes?

Profile request size, rendering time and DOM rows first. Move large-history filtering, sorting and pagination to the API, add suitable indexes and return summaries for charts. Current in-browser operations are appropriate for bounded snapshots, not unlimited historical datasets.

Interview presentation and ownership

Interview questions 76-80

76. What difficult bug can you discuss?

One recorded bug was configuration loading by line scanning, which failed on valid compact JSON. Explain how a real JSON parser and validate-before-replace behavior fixed the underlying assumption. Discuss the compact-JSON regression test. Use this story only after reading and understanding the implementation.

77. How do you describe your contribution honestly?

Separate the starting code, changes you personally made, changes produced with assistance, and code you reviewed and verified. Explain the decisions and tests you can defend. Avoid claiming you independently wrote or discovered every component if that is not true.

78. What should you say when you do not know an answer?

State what you know and the boundary. For example: "I have not measured live saturation yet. My offline benchmark is documented, and I would next measure loss, queue depth, CPU and memory under controlled load." A concrete measurement plan is stronger than a guessed number.

79. What are your next three priorities?

First, independently evaluate classification quality. Second, add a small historical store with retention and query tests. Third, verify optional Windows enforcement in a controlled elevated setup. If performance is the requirement, profile before choosing worker sharding or new infrastructure.

80. How would you summarize the project at the end?

This project demonstrates a complete path from packet bytes to useful local observations, with explicit rules, bounded state and a usable interface. I can explain its class design, thread model and tests. The next step is stronger accuracy and operational evidence, not simply more visual features.

A five-minute interview demo

Time	Action and explanation
0:00-0:40	Give the short pitch. Say whether you are showing synthetic, offline or live traffic.
0:40-1:30	Open Overview. Explain capture-time traffic, protocol counts, top talkers and bounded graph evidence.
1:30-2:20	Open packet explorer, filter by IP/protocol, sort a column and inspect a packet. Explain header fields and metadata-only retention.
2:20-3:10	Open a flow and related packets. Explain the five-tuple, reverse direction and evidence method.
3:10-4:00	Show rules and status. Explain saved, loaded and enforcement; do not claim a synthetic demo blocks real traffic.
4:00-5:00	Show test results, benchmark conditions and one known limitation. Describe a proposed next improvement.

Reproducible commands

npm run demo - start the isolated synthetic demonstration on localhost:3001 by default.
npm test - run engine/API/analytics checks.
npm run test:ui - run browser scenarios.
npm run benchmark:mixed - run the mixed fixture benchmark.
Build the C++ executable first using the instructions in WINDOWS_SETUP.md.

Before presenting

Check the executable path, dependencies and available port. Keep a saved screenshot and an offline fixture in case live capture is unavailable. Never substitute synthetic traffic for real traffic without labeling it. Show only capture data you are authorized to share.

Last-minute revision sheet

Remember	Say it accurately
Language / UI	C++17 engine; Express API; vanilla JS, local Chart.js and SVG.
Live threading	Capture producer, analysis consumer, telemetry exporter and rule reload.
Classification	Visible DNS/HTTP/TLS clues plus heuristics; no HTTPS decryption.
FastPath	Payload/port eligibility check, not a classification cache.
Flow	Bidirectional five-tuple state; retained connections are not open socket count.
Persistence	Latest atomic JSON snapshot; no historical database today.
Limits	10,000 queued frames; 10,000 flows; 300 packet records; 100 recent flow records.
Three kinds of loss	Capture drop, queue drop, parser rejection; policy match is separate.
Rule states	Saved -> loaded acknowledgment; actual OS enforcement needs separate verification.
Browser pause	Stops refreshes, not packet capture.
Recorded tests	14 engine/API checks; 7 browser scenarios; Release build passed.
Recorded performance	100,000 mixed synthetic frames / 2,150 ms; one local offline run.
Remaining work	Independent accuracy measurement, elevated WFP verification, production load evidence.

Practise without notes

Draw the process boundary and thread model. Trace one HTTP packet. Explain queue overload and malformed JSON. Defend the file-based architecture for a local tool. Then design the smallest useful history feature and state how you would test it.

Evidence and code reading map

This guide is grounded in the local repository inspected on 30 September 2026. Recorded test and benchmark outcomes below come from the project audit; generating this PDF does not rerun those checks. Future design answers are explicitly proposals.

Topic	Read in the repository
Pipeline / threads	src/main.cpp; include/packet_queue.h
Source abstraction	include/packet_source.h; src/packet_source.cpp; src/live_capture_source.cpp
Parsing / inspection	src/packet_parser.cpp; src/fast_path.cpp; src/dpi_engine.cpp; DNS, HTTP and SNI parser files
Flow and rules	include/types.h; src/connection_tracker.cpp; src/rule_manager.cpp
Telemetry	src/stats_collector.cpp; src/traffic_analytics.cpp; docs/TELEMETRY.md
Server / health	server.js; lib/health.js
Frontend	public/app.js; public/js/charts.js; explorer.js; network.js; rules.js; dom.js
Verification	PROJECT_AUDIT.md; BENCHMARK.md; tests/; playwright.config.js
Setup / demo	WINDOWS_SETUP.md; DEMO.md; PROJECT_IMPROVEMENT_PLAN.md

Evidence to quote with context

The recorded short live check observed 9 packets, no capture/queue drops or parse errors, and a successful rule revision acknowledgment. WFP was inactive. This supports the tested path only; nine packets cannot establish sustained reliability or firewall enforcement.

Rebuilding this document

Run python scripts/create_interview_guide.py with ReportLab available. The script is documentation tooling, not an application runtime dependency. The editable question content is kept in this script so the PDF can be updated when the project changes.